

JAVA INTERVIEW PREPARATION

CHAPTER 33

SPRING BEAN LIFECYCLE AND SCOPES

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Spring Bean Lifecycle and Scopes: Creation Pipelines, Proxies, and Resource Ownership

Senior / Lead Java Developer Reading Chapter

A Spring bean is not simply constructed and stored. Its definition is resolved, an instance is created, dependencies are populated, awareness and post-processing callbacks run, initialization completes, and infrastructure may return a proxy rather than the raw target. Scope determines when instances are created and destroyed. Understanding this pipeline explains why advice is absent in constructors, why prototype cleanup is special, and why mutable singleton state fails under concurrent requests.

The Lifecycle Mental Model

```
bean definition
-> instantiate
-> populate dependencies
-> aware callbacks
-> BeanPostProcessor before initialization
-> initialization callbacks
-> BeanPostProcessor after initialization
-> exposed bean, possibly a proxy
-> destruction callbacks when owned scope ends
```

The actual container has more extension points, but this order is a useful interview model.

Construction and Dependency Population

Constructor injection resolves collaborators before construction. Setter or field injection occurs after instantiation. Code in a constructor should establish local invariants, not assume every container facility or proxy is active.

```
@Component
final class CatalogClient {
    private final HttpClient client;

    CatalogClient(HttpClient client) {
        this.client = client;
    }
}
```

Avoid remote calls, database migrations, or thread startup in constructors. A constructor failure aborts graph creation and may hide infrastructure readiness behind a `BeanCreationException`.

Awareness Interfaces

Spring can call interfaces such as `BeanNameAware`, `BeanFactoryAware`, and `ApplicationContextAware`. These expose container infrastructure to the bean.

They are appropriate for framework components and integration adapters but are usually a design smell in domain services. If a service needs a `Clock`, `ResourceLoader`, `Environment` value, or event publisher, inject the narrow capability explicitly rather than the entire `ApplicationContext`.

BeanPostProcessor

BeanPostProcessor can inspect or replace bean instances before and after initialization. Annotation injection, method validation, scheduling, asynchronous execution, and AOP auto-proxy creation rely on container post-processing mechanisms.

```
target object -> post-processors -> proxy or decorated object exposed to callers
```

A BeanPostProcessor is itself infrastructure and is created early. Dependencies on ordinary application beans can cause premature creation and make those beans ineligible for later processing. Custom post-processors should be rare, focused, and tested with startup ordering.

Initialization Callbacks

Common choices include `@PostConstruct`, `InitializingBean`, and an explicit `initMethod`.

```
@PostConstruct
void validateConfiguration() {
    if (timeout.isNegative()) {
        throw new IllegalStateException("timeout must be non-negative");
    }
}
```

Prefer `@PostConstruct` or explicit init methods over coupling application code to `InitializingBean`. Initialization is suitable for validation and local preparation. It is a poor place for long remote calls or business processing.

Proxy-based features such as `@Transactional` should not be expected to apply to a method invoked from the same bean's initialization callback. The proxy may not yet be the caller path, and self-invocation bypasses it.

SmartLifecycle and Application Readiness

Lifecycle and SmartLifecycle support controlled start and stop of components such as consumers and schedulers. SmartLifecycle phases define ordering: lower phases start earlier and stop later.

```
start: database-dependent infrastructure -> message consumers
stop: stop consumers -> drain work -> release infrastructure
```

Readiness should become true only after required startup work completes. During shutdown, reject new work, drain bounded in-flight activity, and respect the platform termination grace period. Starting unmanaged threads from arbitrary beans bypasses this lifecycle.

Destruction Callbacks

For singleton beans, context shutdown triggers callbacks such as `@PreDestroy`, `DisposableBean`, and configured destroy methods.

```
@PreDestroy
void close() {
    executor.shutdown();
}
```

Use try-with-resources for short-lived resources and lifecycle callbacks for bean-owned long-lived resources. Shutdown hooks do not run after every form of process termination, so durable correctness cannot depend solely on cleanup.

Singleton Scope Means Per Container

Spring singleton scope provides one instance per bean definition per `ApplicationContext`, not one instance per JVM or class loader.

```
ApplicationContext A -> one PricingService
ApplicationContext B -> another PricingService
```

Singleton beans serve concurrent requests. They should be stateless or protect any mutable state explicitly.

```
@Service
class UnsafeSearchService {
    private String lastQuery; // shared by all request threads
}
```

Method-local variables are naturally request-specific; mutable instance fields are shared. DI scope does not provide thread safety.

Prototype Scope

Prototype scope creates a new instance each time the container resolves that bean. The container performs creation and initialization, but it does not automatically invoke configured destruction callbacks for prototype instances.

```
@Bean
@Scope(ConfigurableBeanFactory.SCOPE_PROTOTYPE)
ReportBuilder reportBuilder() { return new ReportBuilder(); }
```

Injecting a prototype directly into a singleton resolves it once during singleton creation. It does not produce a new instance per method call.

Use `ObjectProvider`, a factory interface, lookup method, or scoped proxy when repeated creation is truly required.

```
ReportService(ObjectProvider<ReportBuilder> builders) {
    this.builders = builders;
}

Report create() {
    return builders.getObject().build();
}
```

The owner that obtains a prototype must own its cleanup.

Web Scopes

In a web-aware `ApplicationContext`, request, session, application, and `WebSocket` scopes bind bean lifetime to corresponding web contexts.

```
singleton -> whole ApplicationContext
request    -> one HTTP request
session    -> one HTTP session
application -> ServletContext
websocket  -> one WebSocket session
```

Session-scoped state can increase memory and complicate horizontal scaling. Prefer stateless servers unless session semantics are required, and store only bounded, serializable or externally managed state according to deployment architecture.

Scoped Proxies

A singleton cannot directly hold the current request instance at startup. A scoped proxy is a stable injected object that resolves the actual scoped target for each invocation.

```
singleton service -> request-scope proxy -> current request bean
```

Proxy identity is not target identity. Avoid synchronizing on scoped proxies, using them as persistent map keys, or expecting target fields through direct reflection. Inject narrow behavior instead of carrying request-scoped data deep into asynchronous work.

Lazy Initialization

`@Lazy` or global lazy initialization defers bean creation until requested. It can reduce startup work, but it moves configuration errors and initialization latency to the first request.

Use laziness for expensive optional components with clear failure handling, not to mask an oversized dependency graph. Critical credentials, migrations, routes, and serializers often deserve eager validation.

FactoryBean Versus BeanFactory

BeanFactory is the container contract. FactoryBean is a bean that creates another object, historically used for complex infrastructure and proxy products. Looking up its bean name returns the produced object; prefixing the name with `&` retrieves the FactoryBean itself.

Most application configuration is clearer with `@Bean` factory methods. Understand FactoryBean because many framework integrations and legacy configurations use it.

Multiple Application Contexts

Some applications have parent and child contexts. Bean visibility can flow from child to parent lookup rules without making both contexts one lifecycle.

Duplicate infrastructure definitions across contexts can create multiple transaction advisors, object mappers, or security components. When a bean appears not to be proxied, verify which context created it and which context registered the post-processor.

Lifecycle Callback Ordering

When several initialization mechanisms are configured, the broad order is annotation callback, framework interface callback, and custom init method. Destruction has a corresponding sequence. Exact behavior can be influenced by inheritance and post-processors, so application correctness should not depend on several callbacks mutating the same state.

```
initialization:
    @PostConstruct -> afterPropertiesSet -> configured init method

destruction:
    @PreDestroy -> destroy -> configured destroy method
```

Choose one application-facing mechanism where possible. Multiple callbacks are more common when integrating legacy types or infrastructure. Callbacks should be idempotent when repeated operational invocation is possible through custom lifecycle code, even though the container normally invokes its standard callback once per managed instance.

DependsOn Is Startup Ordering, Not Business Dependency

`@DependsOn` tells the container that another bean should be initialized first and, for singleton destruction, can influence reverse shutdown order. It does not inject a collaborator or make a hidden runtime dependency visible.

```
@Bean
@DependsOn("schemaInitializer")
ReportRepository reportRepository(dataSource dataSource) { ... }
```

If `ReportRepository` calls `SchemaInitializer` during normal work, inject it explicitly. Use ordering metadata for genuine infrastructure sequencing, such as registration before consumers start, and avoid large chains that make startup fragile.

Early References and Circular Singleton Resolution

Historically, setter or field cycles between singleton beans could sometimes be resolved by exposing an early reference before both beans completed initialization. That mechanism becomes dangerous with proxies: another bean might see a raw or partially initialized object rather than the final exposed proxy.

```
create A -> needs B
create B -> needs A
           receives early A reference
later A may be wrapped with proxy
```

Constructor cycles cannot be solved this way because neither complete instance exists. Modern Boot defaults commonly reject circular references. This is useful: it prevents partially initialized graphs and turns an architectural cycle into a startup error.

Scope Mismatch Is a Lifetime Problem

The deeper rule behind scoped proxies is that a longer-lived object must not directly capture a shorter-lived instance.

```
singleton lifetime: [-----]
request lifetime:   [----]

direct capture is invalid after request ends
proxy/provider resolves current request target on demand
```

The same reasoning applies outside web scopes. A singleton retaining a per-job object, transaction-bound `EntityManager`, or temporary security context creates lifecycle bugs even if annotations compile.

Pass immutable data across lifetime boundaries. For asynchronous work, capture the required tenant ID, locale, or trace context through supported propagation, not the request bean itself.

Scope and Concurrency

Request scope separates instances between requests, but it does not guarantee only one thread accesses an instance. Async dispatch, callbacks, or application code can involve more than one thread during the logical request. Session scope is even more clearly concurrent because one browser can issue parallel requests for the same session.

```
@SessionScope
class ShoppingSession {
    private final List<Item> items = new ArrayList<>(); // concurrent access risk
}
```

Scope controls identity and lifetime, not synchronization. Protect mutable state, use immutable replacement, or store authoritative state in a concurrency-safe external system.

Prototype Is Not Necessarily Short-Lived

Prototype means "the container creates a new instance when requested," not "the instance is temporary" and not "the garbage collector removes it after one method." The requesting owner can retain it forever.

If the prototype owns a file, executor, buffer pool, or client, define an explicit close contract. Often a normal factory returning an `AutoCloseable` resource is clearer than prototype scope:

```
try (ReportSession session = reportSessions.open()) {  
    return session.generate(request);  
}
```

The code makes ownership visible and guarantees cleanup with try-with-resources.

Factory Method Return Types Matter

The declared return type of an `@Bean` method can influence type prediction and when the container can discover implemented interfaces or annotations.

```
@Bean  
PaymentClient paymentClient() {  
    return new ResilientHttpPaymentClient(...);  
}
```

Declaring a useful interface is normally appropriate, but infrastructure that searches for a concrete type may not recognize it as early. Avoid depending on concrete implementation discovery when a contract should be enough; framework integration should document required return-type visibility.

Lifecycle Events Versus Lifecycle Ownership

Context refresh, ready, closed, and availability events are notifications. Receiving an event does not transfer ownership of every resource to the listener. The bean that creates an executor or client should still close it, or delegate ownership to a documented shared facility.

An `ApplicationReady` event is useful for work that must wait until startup completes, but long or failure-prone business processing can delay readiness or disappear after a crash. Durable jobs belong in a scheduler or queue with explicit recovery.

Raw Target, Early Reference, and Final Exposed Bean

Three identities can matter during complex creation: the raw object just constructed, an early reference used to resolve a cycle, and the final object returned after all post-processors.

```
instantiate raw target  
  -> optional early reference for circular singleton resolution  
  -> initialization callbacks  
  -> post-processors may wrap with proxy  
  -> final exposed bean
```

Well-designed application code should normally observe only the final exposed bean. A dependency that captures the raw target early can bypass transactions, security, caching, or validation applied by the final proxy. This is another reason to reject circular designs rather than relying on early-reference mechanics.

Objects created manually with `new`, by an unmanaged framework, or through a factory outside the context do not automatically receive the container's injection, callbacks, and proxying. If infrastructure policy matters, ensure the creation path is container-managed or use a documented integration mechanism.

Destruction Inference and Shared Resources

For `@Bean` methods, Spring can infer common close or shutdown methods in supported configurations, and an explicit `destroyMethod` can document behavior. Do not rely on surprising inference for critical ownership. Make it clear whether a resource is container-owned.

```
@Bean(destroyMethod = "close")
RemoteClient remoteClient(ClientProperties properties) {
    return RemoteClient.connect(properties.endpoint());
}
```

If several beans inject the same singleton client, none of them should close it independently. The context closes it once after dependents stop. For borrowed connections or per-operation sessions, the borrowing code closes each instance through try-with-resources. Scope and ownership must agree.

Destruction order generally follows dependency relationships in reverse so dependents stop before their dependencies. Hidden dependencies created through static access or service location cannot contribute to that ordering, which can cause shutdown calls against an already closed resource.

Request Scope Does Not Cross an Async Boundary

A request-scoped proxy resolves its target using request context associated with the current execution. Submitting work to another executor does not automatically copy that context, and the HTTP request may end before the task runs.

```
HTTP thread -> request bean available
|
+-> enqueue task -> later worker -> no active request scope
```

Capturing the proxy is not the same as capturing a stable target. Pass immutable values such as tenant ID, authenticated subject identifier, locale, or correlation ID using supported context propagation. Do not retain the servlet request or a request bean inside a long-lived task.

Graceful Shutdown Is a Bounded Protocol

Shutdown should stop admission, stop consumers from fetching new work, drain or checkpoint bounded in-flight work, and release dependencies before the platform deadline.

```
readiness false
-> stop new traffic/consumption
-> wait for bounded in-flight work
-> cancel or persist remaining work by policy
-> close clients, pools, and context
```

`SmartLifecycle` phases can order components, but they cannot make an unbounded task finish. Every component needs a timeout and a defined outcome for unfinished work. If message processing is at-least-once, shutdown may intentionally leave a message unacknowledged for redelivery; if a local batch has no durable checkpoint, termination can lose progress.

Liveness and readiness have different meanings. A process can be alive while not ready for traffic during startup, overload, or draining. Do not tie liveness to a transient database failure and trigger restart storms unless process recovery genuinely requires restart.

Measuring Lifecycle and Startup

Slow startup should be decomposed into configuration discovery, bean-definition processing, post-processor registration, singleton instantiation, migrations, server startup, runners, and readiness. Spring's `ApplicationStartup` abstraction and Boot startup endpoint support can expose startup steps when configured.

Instrumentation must not log secrets or every bean at excessive volume. Focus on the slowest phases and unexpected eager creation. A bean marked lazy can still be created early because an eager singleton depends on it or an infrastructure processor performs type discovery.

For shutdown, record phase start/end, in-flight work, timeout, forced cancellation, and unclosed resources. A termination that always consumes the full grace period is an operational defect even if the process eventually exits.

A Lifecycle Review Checklist

```
construction: local and deterministic?
initialization: bounded, observable, and safe before proxy use?
scope: matches the data lifetime?
concurrency: safe for every thread that can access the instance?
ownership: exactly one closer for each resource?
shutdown: ordered, timed, and recoverable?
async boundary: immutable context copied explicitly?
```

This checklist is more useful than memorizing callback names because it connects lifecycle to production behavior.

A Strong Interview Explanation

When asked whether Spring singleton beans are thread-safe, the correct answer is no. Singleton scope defines one identity per bean definition per container. Because request threads share that identity, thread safety depends on the bean's state and synchronization design. Stateless services are naturally safe when their collaborators are also safe.

When asked why prototype destruction is not automatic, explain ownership: after returning a prototype, the container no longer knows how long the client retains it. Automatically destroying it at an arbitrary time would be incorrect. The obtaining code must manage cleanup, or a custom scope must define an end-of-life boundary.

Production Perspective

Startup hangs often originate in remote work during initialization, circular dependencies, or lifecycle components waiting for unavailable infrastructure. Shutdown hangs often involve non-daemon executors, consumers that ignore interruption, or phase ordering.

Memory incidents can come from session scope, prototype instances whose owner never closes them, long-lived scoped proxies retaining context, and singleton caches without bounds. Inspect bean scope and ownership before increasing memory.

Interview-Ready Summary

Spring creates, populates, post-processes, initializes, and may proxy a bean before exposing it. Initialization callbacks should validate and prepare local state; proxy-based advice should not be assumed inside construction or self-invocation. Destruction runs for container-owned scopes when the context shuts down.

Singleton means one instance per bean definition per container and does not imply thread safety. Prototype destruction is the caller's responsibility. Web scopes bind instances to request or session lifecycles, often through scoped proxies. Lifecycle design must include startup readiness, graceful shutdown, and resource ownership.

Revision Notes

- Bean creation includes instantiation, injection, post-processing, and initialization.
- The exposed bean may be a proxy rather than the raw target.
- Constructors should establish local invariants, not perform remote startup work.
- Awareness interfaces expose container infrastructure and should be used narrowly.
- BeanPostProcessor can inspect or replace beans.
- `@PostConstruct` is appropriate for validation and local preparation.
- Do not rely on proxy advice during bean initialization.
- SmartLifecycle coordinates ordered startup and shutdown.
- Destruction callbacks manage bean-owned long-lived resources.
- Singleton means one instance per definition per ApplicationContext.
- Singleton beans are not automatically thread-safe.
- Prototype creates new instances on resolution.
- Prototype destruction is not automatically managed by the container.
- Direct prototype injection into a singleton occurs once.
- ObjectProvider can obtain a new prototype when needed.
- Web scopes follow request, session, application, or WebSocket lifetime.
- Scoped proxies resolve the current scoped target at invocation time.
- Lazy initialization defers failures and first-use cost.
- FactoryBean creates an exposed product; BeanFactory is a container API.
- Context boundaries affect visibility, proxying, and lifecycle.
- Application code should normally observe only the final post-processed bean.
- Manual construction does not automatically receive container lifecycle or proxying.
- Destruction order depends on visible bean dependencies and ownership.
- Request-scoped proxies do not make request context survive asynchronous execution.
- Graceful shutdown must be bounded and define unfinished-work behavior.
- Readiness can be false while the process remains correctly alive.
- Measure startup and shutdown phases instead of treating lifecycle as one duration.